

Egyptian E-Learning University
Faculty of Computers & Information Technology

NileGuide

Tourism Planning & Recommendation Platform

Final Graduation Project Documentation

Team Members

Name	ID
Yossef Abdelmoneim Abdel hamid	2200395
Abdelrahman Tamer Mahmoud	2200406
Mohamed Ahmed Attia	2200401
Youssef Mohamed Ali	2200718
Ziad hamdy mohamed	2202656
Zyad Samy Mohamed	2200399
Ahmed Hassan Younes	2202660
Ahmed Hesham Abdelshakour	2200783

Table 1. NileGuide team members

Supervised by
Dr. Kamal Hamza
Assistant: Eng. Mohamed Kamal
Academic Year 2025-2026

Abstract

NileGuide is a web-based tourism planning platform built to help tourists discover activities in Egypt, compare activity information, save preferred options, create a personal schedule, view planned places on a map, and ask questions through an AI knowledge assistant. The final system includes two major sides: a tourist website and an administrator dashboard. The tourist side covers activity browsing, filtering, dynamic activity details, wishlist, schedule, map, profile, contact, authentication and chatbot features. The administrator side covers user management, activity management, reports and AI knowledge-file management.

The project was implemented using Angular Standalone for the frontend, ASP.NET Core 8 Web API for the main backend, SQL Server with Entity Framework Core for relational storage, Cloudinary for uploaded images, and a separate FastAPI-based RAG service for the AI knowledge assistant. MongoDB stores AI file metadata and chunk information, while Qdrant stores vector embeddings used for semantic retrieval. The final delivered scope does not present payment, real booking confirmation, Remember Me or refresh-token UX as completed features. They are treated as future work. This report documents the final architecture, implementation, database, APIs, workflows, user interface, testing and deployment considerations of the delivered system.

Document Scope

This document is written as the final project documentation, not only as a short demo report. It is based on the final source code, the delivered screenshots and the original midyear plan. Sections that were planned in the first term but were not delivered as final features are not described as completed. The screenshots are placed first, followed immediately by their explanations, so the reader can connect the visual screen with the actual user behavior and implementation.

Table of Contents

- Chapter 1 - Introduction
- Chapter 2 - Background and Related Work
- Chapter 3 - Requirements and Scope
- Chapter 4 - System Architecture and Design
- Chapter 5 - Database Design
- Chapter 6 - Backend Implementation
- Chapter 7 - Frontend Implementation
- Chapter 8 - AI/RAG Implementation
- Chapter 9 - Graphical User Interface and Feature Walkthrough
- Chapter 10 - API Documentation
- Chapter 11 - Testing and Validation
- Chapter 12 - Deployment and Operations
- Chapter 13 - Challenges and Solutions
- Chapter 14 - Conclusion and Future Work
- Appendices

List of Abbreviations

Abbreviation	Meaning
API	Application Programming Interface

AI	Artificial Intelligence
RAG	Retrieval-Augmented Generation
JWT	JSON Web Token
DTO	Data Transfer Object
EF Core	Entity Framework Core
SPA	Single Page Application
SSR	Server-Side Rendering
CRUD	Create, Read, Update, Delete
LLM	Large Language Model
Vector DB	Database optimized for vector similarity search
UI/UX	User Interface / User Experience

Table 2. List of abbreviations

How to Read This Documentation

The report is organized from the general idea to implementation details. The first chapters explain the problem, requirements and architecture. The middle chapters explain the database, backend, frontend and AI/RAG service. The GUI chapter shows each screenshot first and then explains what the user or administrator does on that screen. The final chapters document APIs, testing, deployment, challenges and future work.

During discussion, the most important parts to focus on are the system architecture, the user workflow, the admin workflow, the database relationships, the dynamic activity-details route, the JWT-based authorization flow and the RAG file-upload/chat flow. These are the parts that connect the project as a full-stack system.

Chapter 1 - Introduction

1.1 Project Overview

NileGuide is a tourism planning and recommendation platform focused on Egyptian travel experiences. It gives the tourist a single place to discover activities, view details, save interesting options, build a schedule, and view planned activities on a map. It also gives the administrator a controlled dashboard for managing users, activities, reports and AI knowledge files.

The first-term idea described NileGuide as a tourism planning and recommendation platform. During the final implementation, the team added concrete web functionality around activity management, profile preferences, wishlist, schedule, map visualization, report screens and an AI/RAG assistant. Some ideas from the first-term plan, such as direct booking or payment, remained outside the final delivered scope and are therefore documented as future work instead of completed modules.

1.2 Motivation

- Tourists usually collect trip information from multiple sources such as blogs, social media, Google Maps, booking pages and personal recommendations. This creates scattered decisions and extra effort.
- A tourism website should not only show a list of places. It should help the tourist move from discovery to planning through wishlist, schedule and map tools.
- Tourism data changes over time. Administrators need a dashboard to add, update, hide or delete activity data without editing the frontend code.
- AI support should be maintainable. NileGuide uses uploaded knowledge files so the assistant can answer from controlled content rather than relying only on generic model knowledge.

1.3 Problem Definition

The main problem addressed by NileGuide is the lack of a unified web experience for discovering and planning Egyptian activities. Many tools show information, but they do not always connect activity details with wishlist, itinerary scheduling, map visualization, profile preferences and administrator control. NileGuide combines these parts into a single project that can be demonstrated and maintained.

Problem area	Description	NileGuide response
Scattered activity information	Tourists often need to compare several websites before choosing an activity.	A single activity catalogue with city, category, price, images and details.
Weak planning flow	Users can discover attractions but still need separate notes or apps to organize their day.	Wishlist, schedule and map modules connect discovery with trip planning.
Content maintenance	Static pages become outdated when activities, images or status change.	Admin activity management allows controlled CRUD operations and image upload.
Uncontrolled AI answers	Generic chatbots may answer without using project-approved content.	RAG assistant retrieves from uploaded knowledge files before answering.

Table 3. Problem definition and project response

1.4 Project Objectives

- Build a responsive Angular frontend for tourists and administrators.

- Provide secure authentication and role-based access for Tourist and Admin roles.
- Load real activity, city, category, wishlist, schedule, profile and report data from APIs.
- Implement an admin dashboard for operational control over users, activities, reports and AI files.
- Integrate an AI/RAG service that allows project knowledge to be updated through file uploads.
- Deploy the system in a way that separates frontend, backend and AI responsibilities.

1.5 Delivered Scope and Out-of-Scope Items

Area	Delivered in final system	Not included in final delivered scope
Authentication	Register, login, protected routes and JWT access-token usage.	Remember Me and user-visible refresh-token flow.
Tourist planning	Activities, details, wishlist, schedule, map and profile preferences.	Automatic paid booking or payment confirmation.
Admin panel	Users, activities, reports and AI files/chatbot UI.	Advanced audit logs and enterprise admin permissions.
AI module	RAG file upload/list/update/delete and chat answers from retrieved content.	Fully personalized automated itinerary generation as a final guaranteed feature.

Table 4. Final project scope

1.6 Final Contributions

The final implementation contributes a complete working flow rather than isolated screens. A tourist can start at the public home page, authenticate, browse activities, open details, save interests, build a schedule, view locations on a map and ask the AI assistant. An administrator can maintain the same platform through a dashboard instead of relying on code changes or static content.

Contribution	Why it matters
Dynamic activity catalogue	Activities are loaded and managed through APIs, which makes the platform maintainable.
Integrated planning flow	Wishlist, schedule and map are connected to activity discovery instead of being separate tools.
Operational admin dashboard	The admin can manage users, activities, reports and AI knowledge from the interface.
Grounded AI assistant	The RAG service answers from uploaded knowledge files, which makes the AI content updateable.
Documented final scope	The report separates delivered modules from future work so the discussion remains accurate.

Table 5. Final project contributions

1.7 Implementation Reality Compared with First-Term Plan

The first-term document described the platform idea, related work, initial architecture and expected work plan. In the final version, several parts became concrete implemented modules: Angular pages,

ASP.NET Core APIs, SQL Server entities, profile and wishlist flows, admin activity management, reports and the RAG assistant. At the same time, some early ideas were intentionally postponed, such as direct payment, automatic booking confirmation and advanced fully personalized itinerary generation.

Chapter 2 - Background and Related Work

2.1 Tourism Planning Background

Tourism planning involves more than choosing a destination. A tourist needs to know what activities exist, where they are located, how long they take, how much they cost, whether they fit the available trip time, and whether they match personal preferences. In Egypt, this is especially important because activities can be spread across different cities and may vary by provider, schedule and price.

NileGuide approaches the problem from a practical software-engineering point of view. The platform does not attempt to replace official booking services. Instead, it organizes information and provides a planning workflow that can later be connected to external providers or more advanced recommendation models.

2.2 Related Work Summary

Direction	Common approach	Limitation	NileGuide decision
Itinerary planning systems	Generate plans using scoring rules based on distance, time and popularity.	Often focus on optimization but provide limited admin and content-management workflow.	Give the tourist manual control through wishlist and schedule while keeping the catalogue manageable.
Tourism recommender systems	Recommend places based on user preferences, images, ratings or hybrid filtering.	May depend on limited datasets and do not always include a full deployed web system.	Use preferences and activity filters now, with AI and stronger recommendations as future expansion.
Travel chatbot systems	Answer trip questions using NLP or scraped data.	Risk of outdated or unsupported answers if the knowledge source is uncontrolled.	Use RAG with admin-uploaded files and retrieved context.
Booking aggregators	Connect users to flights, hotels or package providers.	Booking/payment logic increases complexity and needs external agreements.	Show provider links but keep booking confirmation outside the final scope.

Table 6. Related work comparison

2.3 Research Gap

- Many tourism prototypes solve one problem only: recommendations, itinerary generation or attraction listing. NileGuide connects discovery, planning, administration and AI knowledge updates.
- Several systems evaluate recommendation accuracy but do not discuss day-to-day platform operations such as content CRUD, image upload, reports and roles.
- Generic chatbots can answer tourism questions but are difficult to keep aligned with project data. The RAG service solves this by allowing the admin to upload the knowledge source.
- A graduation project needs a demonstrable, deployable system. NileGuide prioritizes a working platform with clear user and admin flows.

2.4 Value Added by NileGuide

The value of NileGuide is not only in listing activities. The platform provides continuity: a tourist can move from home page to activities, from activity details to wishlist, from wishlist to schedule, and from schedule to map. The administrator can maintain the same content from the dashboard, and the AI assistant can be updated by uploading files. This makes the project complete enough to be discussed as a full-stack system rather than a static website.

Chapter 3 - Requirements and Scope

3.1 Stakeholders

Stakeholder	Responsibilities / interaction
Tourist / Registered user	Uses the public platform, browses activities, saves wishlist items, schedules activities, edits profile and asks the chatbot.
Administrator	Accesses dashboard modules, manages users and activities, reviews reports and maintains AI knowledge files.
Project team	Develops and maintains frontend, backend, database, AI/RAG and deployment.
Supervisor / evaluator	Reviews final implementation, documentation, demo flow and engineering decisions.

Table 7. System stakeholders

3.2 Functional Requirements

ID	Requirement	Final implementation
FR-01	User registration	Create account using required personal information and credentials.
FR-02	User login	Authenticate using email and password and store a JWT access token.
FR-03	Role-based access	Allow Tourist and Admin users to reach different protected routes.
FR-04	Activity browsing	Load activities from the backend and display them in a card-based interface.
FR-05	Filtering and search	Filter activities by city, category, status, price or keyword depending on the page.
FR-06	Activity details	Show complete dynamic activity details using /activities/:id.
FR-07	Wishlist	Allow a tourist to save, view, remove and clear saved activities.
FR-08	Schedule	Allow a tourist to organize selected activities into a personal itinerary.
FR-09	Map	Display planned activities and nearby planning information on a

		map.
FR-10	Profile	Edit user information, travel dates, preferred cities and interests.
FR-11	Reviews	Display and create reviews for activities where supported.
FR-12	Admin users	List, filter, view, update status/role and manage user accounts.
FR-13	Admin activities	Create, update, delete and filter activity records and images.
FR-14	Reports	Display user growth, activity views, category distribution and top activities.
FR-15	AI files	Upload, list, update and delete files used by the RAG assistant.
FR-16	AI chat	Send questions and receive answers grounded in retrieved project knowledge.

Table 8. Functional requirements

3.3 Non-Functional Requirements

Quality attribute	Requirement	Implementation notes
Usability	The interface should be clear and usable for non-technical tourists.	Dark UI, cards, icons, filters, labels, modals and readable typography.
Security	Protected operations must require authenticated users.	JWT Bearer token, role guards and backend authorization attributes.
Maintainability	Modules should be separated and easier to modify.	Angular services/components, backend controllers/services/DTOs and separate AI service.
Responsiveness	Pages should work on common screen sizes.	Tailwind utility classes and responsive grids.
Extensibility	The platform should support adding new cities, categories, activities and knowledge files.	Lookup APIs, admin CRUD and RAG file management.
Reliability	The user should get loading/error states instead of silent failures.	Frontend state handling, API error handling and testing workflow.

Table 9. Non-functional requirements

3.4 Final User Stories

- As a tourist, I want to browse activities by city and category so that I can quickly find activities that match my trip.

- As a tourist, I want to open activity details so that I can see photos, description, documents, provider links and reviews before deciding.
- As a tourist, I want to save activities to my wishlist so that I can return to them later.
- As a tourist, I want to add activities to a schedule so that I can prepare a clear itinerary.
- As a tourist, I want to use the map so that I can understand where planned activities are located.
- As an admin, I want to manage activities and images so that the catalogue stays updated.
- As an admin, I want to upload AI files so that the chatbot can answer from updated knowledge.

Chapter 4 - System Architecture and Design

4.1 High-Level Architecture

NileGuide System Architecture

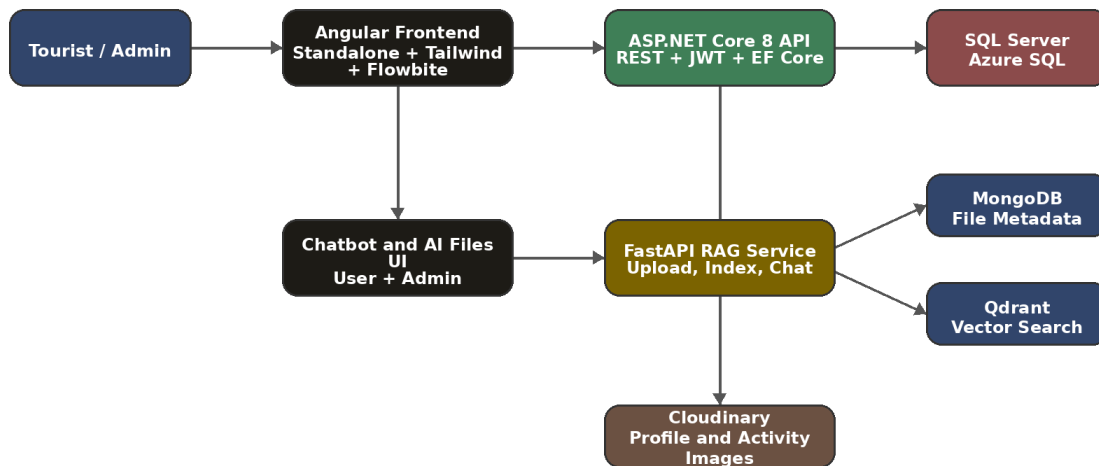


Figure 1. Final NileGuide architecture

The architecture is divided into three runtime parts. The Angular frontend handles user interaction, routing and UI state. The ASP.NET Core backend handles authentication, business APIs, database operations and image integration. The FastAPI RAG service handles AI file processing, embeddings, vector search and chat answers. This separation keeps the project easier to deploy, debug and explain.

4.2 Architectural Layers

Layer	Responsibility	Main technologies
Presentation layer	Pages, components, routing, guards, forms, modals, charts and chatbot UI.	Angular Standalone, Tailwind CSS, Flowbite, ApexCharts, jsPDF
Application/API layer	REST APIs, validation, authentication, role authorization, dashboard, reports and business services.	ASP.NET Core 8, EF Core, JWT, Swagger
Data layer	Relational storage for users, activities, wishlist, schedule, reviews and reports.	SQL Server / Azure SQL, Entity Framework Core
Media storage	Stores uploaded profile and activity images outside the database.	Cloudinary
AI knowledge layer	File ingestion, chunking, embeddings, vector search and LLM response generation.	FastAPI, MongoDB, Qdrant, OpenAI-compatible models

4.3 Tourist Workflow

Tourist Workflow in the Delivered System

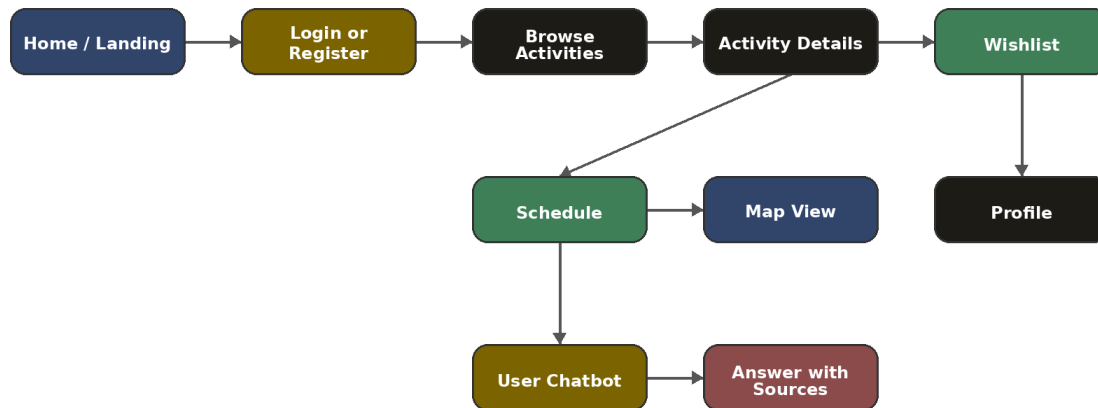


Figure 2. Tourist workflow

The tourist flow starts with the home page and continues to login/register when protected features are needed. After authentication, the tourist can browse activities, open details, save to wishlist, add activities to a plan, view the map, update profile preferences and use the chatbot. The flow keeps decisions under user control instead of forcing an automatically generated itinerary.

4.4 Admin Workflow

Admin Workflow

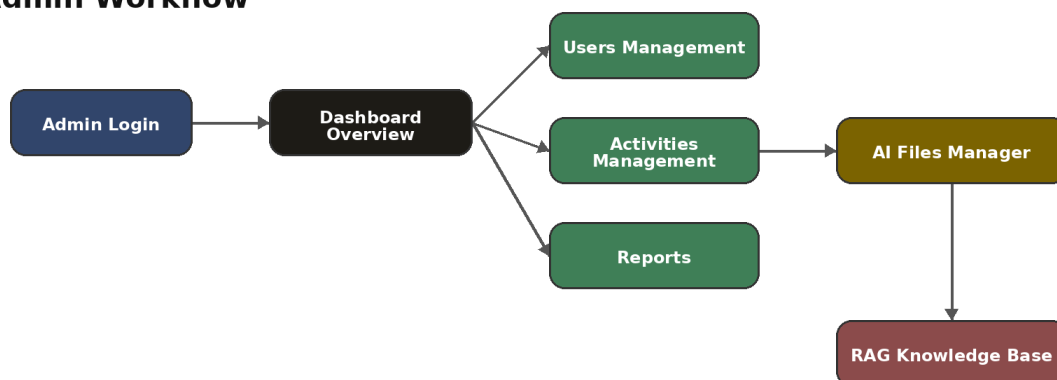
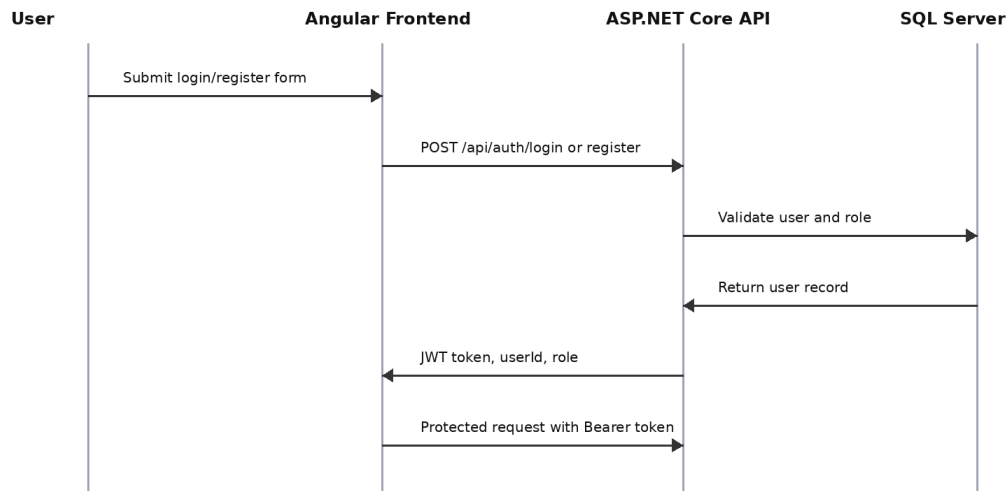


Figure 3. Admin workflow

The admin workflow is built around dashboard modules. The admin reaches users management, activities management, reports and AI file management from the dashboard shell. This design makes operational actions separate from tourist actions and helps protect sensitive features.

4.5 Authentication Sequence

Authentication and Authorization Sequence



Final report scope: JWT access token and role-based route protection. Remember Me and refresh-token UX are not treated a

Figure 4. JWT authentication sequence

The final authentication flow uses JWT access tokens. After login, the backend returns a token and role. The frontend stores the token and sends it in the Authorization header for protected requests. Admin and tourist routes are protected on the frontend by guards and on the backend by authorization rules. Remember Me and refresh-token UX are not described as delivered features in the final scope.

4.6 Deployment Architecture

Deployment View

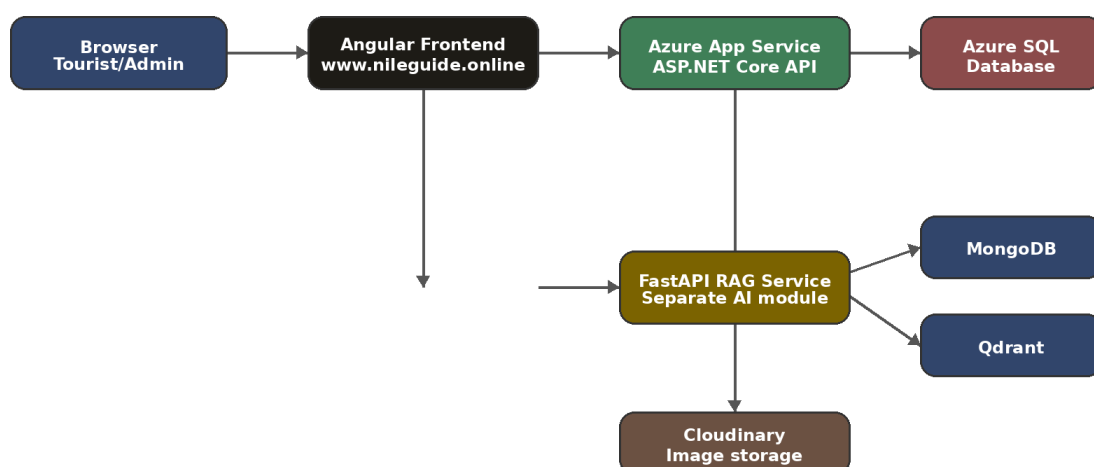


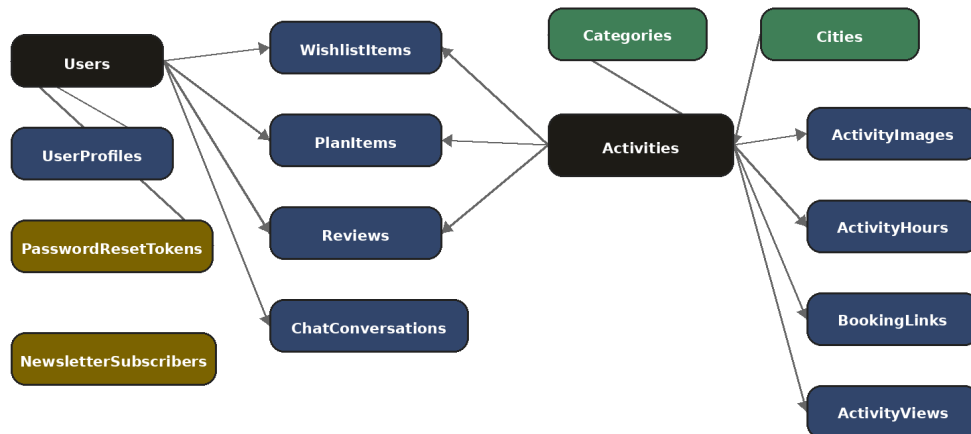
Figure 5. Deployment architecture

The deployment separates the public frontend, the .NET API and the AI service. The backend can be deployed on Azure App Service and connected to Azure SQL. The frontend can be served under the public NileGuide domain. The AI/RAG service can be deployed separately because its dependencies and storage are different from the .NET backend.

Chapter 5 - Database Design

5.1 Entity Relationship Overview

Core Database Design



The diagram shows the relationships used by discovery, profile, wishlist, schedule, review, reporting, and chat modules. Refresh tokens exist in t

Figure 6. Simplified ERD for NileGuide

The database is relational because most platform data has clear relationships: users own profiles, wishlist items, plan items and reviews; activities belong to cities and categories; activities have images, hours, booking links and reviews. The RAG service uses its own MongoDB and Qdrant storage because document chunks and vectors are not relational data.

5.2 Main Entities

Entity	Main purpose	Important fields / relationships
Users	Stores account identity, credentials and role.	UserId, FullName, Email, Role, IsActive; linked to profile, wishlist, plan, reviews.
UserProfiles	Stores additional tourist preferences and personal information.	Nationality, DateOfBirth, ProfilePictureUrl, TravelStartDate, TravelEndDate, PreferredCityIds, InterestCategoryIds.
Activities	Stores the main tourism activity data shown to tourists.	ActivityID, ActivityName, Description, CategoryID, CityID, Price, Duration, IsActive, Rating, Latitude, Longitude.
Categories	Groups activities by type.	CategoryID, CategoryName; used in filters and activity details.
Cities	Groups activities by destination.	CityID, CityName; used in filters,

		details and map context.
ActivityImages	Stores image URLs for activity galleries.	ActivityID, ImageUrl, Cloudinary/public image information.
ActivityHours	Stores opening and closing time information.	ActivityID, Day/Time fields or structured hour values.
BookingLinks	Stores external provider links.	Provider name and URL for sources such as activity providers.
WishlistItems	Connects users with saved activities.	UserId, ActivityId.
PlanItems	Stores activities selected for a schedule.	UserId, ActivityId, SelectedDate, StartTime and calculated itinerary data.
Reviews	Stores user ratings and written reviews.	UserId, ActivityId, Rating, Comment, CreatedAt.
ActivityViews	Stores analytics for reports.	ActivityId, ViewDate or count aggregation.
ChatConversations	Stores conversation identifiers/history metadata used by chat history.	UserId, ConversationId and timestamps.

Table 11. Database entity dictionary

5.3 Relationship Notes

- One user can have one profile and many wishlist items, plan items, reviews and chat-conversation records.
- One city and one category can be assigned to many activities.
- One activity can have many images, opening-hour records, booking links, wishlist links, plan links, reviews and view records.
- WishlistItems and PlanItems act as join/relationship tables between users and activities.
- RAG files are intentionally outside the SQL schema because they are stored and indexed by the AI service.

5.4 Data Integrity Considerations

The database design supports normalized lookup data for cities and categories, preventing repeated text values across activities. Activity-related tables keep images, hours and booking links separate so the activity record does not become overloaded. User-generated data such as wishlist, plan and reviews is stored separately, which allows the platform to remove or update user-specific records without changing the public activity catalogue.

Chapter 6 - Backend Implementation

6.1 Backend Technology Stack

Technology	Usage in NileGuide
ASP.NET Core 8 Web API	Main backend framework for REST endpoints and application services.
Entity Framework Core	Object-relational mapping and database access.
SQL Server / Azure SQL	Relational database for platform data.
JWT Authentication	Access-token based authentication for protected APIs.
Role-based Authorization	Separates tourist and admin operations.
Cloudinary	Image upload and storage for profile pictures and activity images.
Swagger / OpenAPI	API exploration and testing during development.
SMTP / Email settings	Used by password reset and email-related flows.

Table 12. Backend technology stack

6.2 Backend Code Organization

Folder / part	Role
Controllers	Expose REST endpoints and receive HTTP requests.
Services	Contain business logic and coordinate database operations.
DTOs	Define clean request and response shapes used by APIs.
Models / Entities	Represent database tables and relationships.
Data / ApplicationDbContext	Maps entities and database configuration.
Middleware	Centralized error handling and request pipeline behavior.
Options / Configuration	JWT, Cloudinary, Email and connection-string settings.

Table 13. Backend organization

6.3 Controllers and Responsibilities

Controller	Responsibility
AuthController	Register, login, current user, password reset and logout-related endpoints.
ActivitiesController	Public activity list and activity details.

AdminActivitiesController	Admin activity CRUD with multipart/form-data and image upload.
CategoriesController / CitiesController	Lookup data used by filters and profile preferences.
WishlistController	Tourist wishlist list, status, add and remove operations.
PlanController	Tourist itinerary/schedule items and saved activity IDs.
ReviewsController	Activity-specific reviews and rating submissions.
UserProfilesController	Logged-in tourist profile retrieval and update.
UsersController	Admin user management and profile-picture operations.
DashboardController	Admin dashboard summary statistics.
ReportsController	Admin analytical reports for charts.
ChatConversationsController	Stores/retrieves conversation metadata used by chatbot history.
NewsletterController	Newsletter subscription and sending operations.

Table 14. Backend controllers

6.4 Backend Feature Flows

Feature	Backend process
Login	Validate credentials, create JWT token, return role and user information.
Activity list	Accept optional filters/search/paging and return a paged list of activity DTOs.
Activity details	Load one activity with images, hours, booking links, location and review data.
Admin create/update activity	Receive form-data, validate fields, upload images and save activity-related records.
Wishlist	Check authenticated user, insert or remove relationship between user and activity.
Schedule	Store selected activity in user plan and return itinerary data.
Profile picture	Receive image file, upload to Cloudinary and save returned URL.
Reports	Aggregate database records into chart-friendly DTOs.

Table 15. Backend feature processing summary

6.5 Security Implementation

- Protected APIs require a valid Bearer token in the Authorization header.
- Admin endpoints are restricted to users with the Admin role.
- Tourist-specific endpoints use the authenticated user identity instead of accepting arbitrary user IDs from the client.
- Entity Framework Core and parameterized queries reduce SQL injection risk.
- Passwords are not stored as plain text; password handling is managed through backend authentication logic.
- Refresh-token and Remember Me behavior are not part of the documented final delivered user flow.

Chapter 7 - Frontend Implementation

7.1 Frontend Technology Stack

Technology	Usage
Angular 21 Standalone	Main frontend framework and route/component structure.
Angular Router	Public, auth, tourist and admin navigation.
Tailwind CSS	Utility-based responsive styling.
Flowbite / Flowbite Datepicker	UI components and date-picker interactions.
RxJS	HTTP subscriptions, state updates and reactive behavior.
ApexCharts	Dashboard/report charts.
jsPDF / autotable	Schedule or report export features.
Font Awesome	Icons in navigation, buttons and cards.
SSR/Hydration support	Angular server-side rendering structure included in the project.

Table 16. Frontend technology stack

7.2 Frontend Routes

Route	Purpose	Access
/home	Landing page and platform introduction.	Public
/auth/login	User/admin login.	Public
/auth/register	Tourist registration.	Public
/auth/forgot-password	Start password recovery.	Public
/auth/reset-password	Complete password reset after code verification.	Public
/activities	Browse and filter activities.	Tourist/Public depending on guard configuration
/activities/:id	Dynamic activity details page.	Tourist/Public depending on guard configuration
/wishlist	Saved activities.	Tourist
/schedule	Personal itinerary/schedule.	Tourist
/map	Map visualization of planned activities.	Tourist

/profile	User profile and preferences.	Tourist
/contact	Contact form and project communication.	Public
/dashboard	Admin dashboard shell.	Admin
/dashboard/users-management	Admin users module.	Admin
/dashboard/activities-management	Admin activities module.	Admin
/dashboard/reports	Admin reports module.	Admin

Table 17. Frontend route map

7.3 Frontend Services

Service / module	Main responsibility
AuthService	Login, register, token storage and logout behavior.
ActivitiesService	Load activities, details, categories and cities.
WishlistService	List saved activities, add/remove items and check wishlist status.
Plan/Schedule service	Add, list and delete schedule items and activity IDs.
ProfileService	Load/update profile and upload/delete profile picture.
Admin Activities service	Create, update and delete activity records using admin API.
UsersService	Admin user listing and user actions.
ReportsService	Load report datasets used by charts.
Chatbot / AI service	Send chat questions and manage AI file operations.

Table 18. Frontend services and responsibilities

7.4 State and UI Handling

The Angular implementation uses component state variables for loading, errors, selected filters, modal status, pagination and selected records. HTTP services isolate API calls away from templates. Route guards protect admin and tourist areas. Interceptors attach authentication headers and handle request behavior. The UI uses modals for add/update/delete flows and cards/tables for listing data.

7.5 Frontend Design Decisions

- Angular Standalone structure was used to reduce module boilerplate and keep feature pages clearer.
- Tailwind CSS was used for consistent dark interface styling and responsive grids without large separate CSS files.
- Dynamic API data replaced static content in important pages such as activities, details, wishlist and profile.
- Profile and activity image handling uses real file upload instead of only URL input.

- Admin activity management uses modals so the admin can perform actions without leaving the management page.
- The chatbot is displayed as a floating panel to keep it available without changing the current route.

Chapter 8 - AI/RAG Implementation

8.1 AI Module Overview

RAG File Processing and Chat Flow

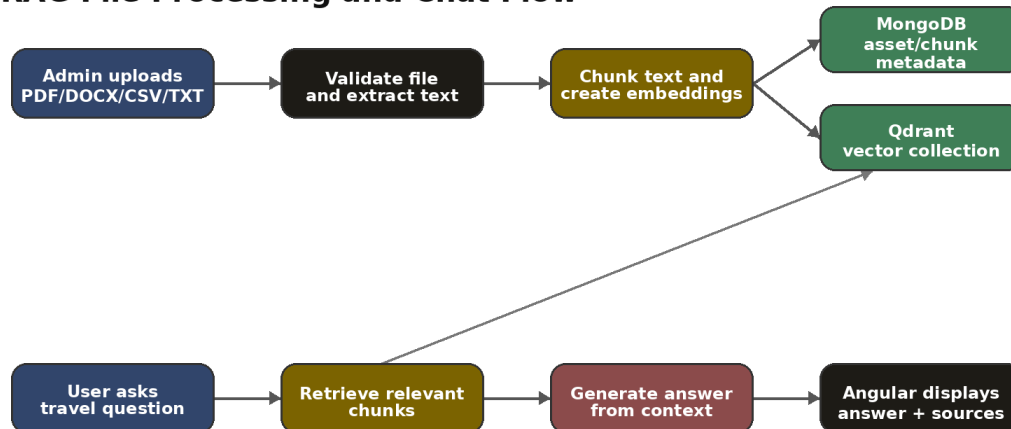


Figure 7. RAG workflow for NileGuide AI assistant

The AI part of NileGuide is implemented as a separate FastAPI service. It is not a generic chatbot only. The system allows the administrator to upload files, extract their text, split the text into chunks, generate embeddings, store metadata and vectors, then answer user questions by retrieving relevant chunks and sending them to a language model.

8.2 AI/RAG Technology Stack

Component	Role
FastAPI	Exposes AI file-management and chat endpoints.
MongoDB	Stores project/file metadata and text chunks.
Qdrant	Stores embeddings and performs vector similarity search.
OpenAI-compatible embeddings	Convert chunks into vectors.
OpenAI-compatible generation model	Generates answers from retrieved context.
Text extraction modules	Read TXT, PDF, DOCX and CSV content.

Table 19. AI/RAG technology stack

8.3 RAG Data Flow

- Admin uploads a supported file from the admin AI files panel.
- The FastAPI service validates file type and size.
- The service extracts text from PDF, DOCX, CSV or TXT formats.
- Extracted text is split into smaller chunks to make retrieval accurate.

- Metadata and chunks are stored in MongoDB.
- Embeddings are generated and stored in Qdrant.
- When a user asks a question, the service retrieves the most relevant chunks and uses them as context for the answer.

8.4 AI Endpoints

Endpoint	Method	Purpose
/	GET	Health/root endpoint for the AI service.
/api/v1/data/multi-upload	POST	Upload one or more knowledge files.
/api/v1/data/files	GET	List uploaded knowledge files.
/api/v1/data/file/{file_id}	PUT	Update an existing AI file.
/api/v1/data/file/{file_id}	DELETE	Delete an AI file and related indexed data.
/api/v1/nlp/chat	POST	Ask a question and receive a RAG-based answer.

Table 20. AI/RAG endpoints

8.5 Why AI is Separate from the .NET Backend

The AI service has a different technical nature from the main backend. It depends on document extraction, vector databases, embeddings and LLM calls. Keeping it separate avoids mixing heavy AI dependencies with the .NET business API. It also allows the team to scale or replace the AI service without rewriting the core platform backend.

Chapter 9 - Graphical User Interface and Feature Walkthrough

9.1 User Interface Style

The interface uses a dark visual theme, large cards, gold/yellow accent colors and clear action buttons. The design goal is to make tourism content visually attractive while keeping forms and admin tables readable. The following sections place each screen first and then explain exactly what the user or administrator can do on that screen.

9.2 Public and Authentication Screens

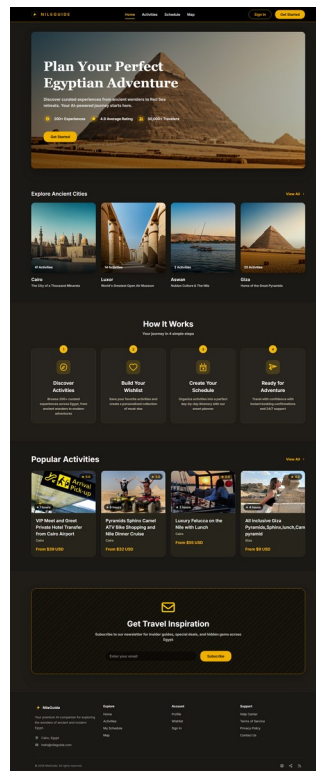


Figure 8. Home page

Purpose	Introduces NileGuide and directs visitors to explore Egyptian destinations and activities.
What appears on the screen	Hero section, action buttons, platform statistics, city cards, how-it-works section, popular activities and FAQ/contact sections.
User/Admin action	The visitor can start exploring, sign in, move to activities or read platform information.
System behavior	This screen is mainly frontend presentation, but it links users to dynamic activity and authentication flows.

Table 21. Home page explanation

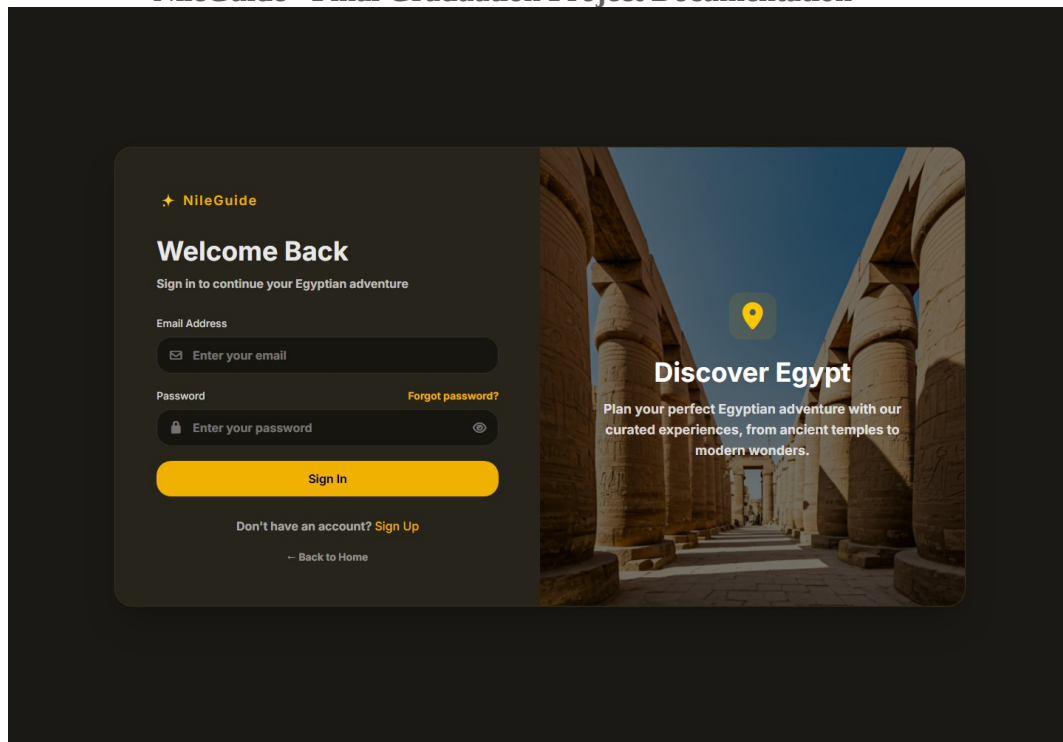


Figure 9. Login page

Purpose	Authenticates existing users and admins.
What appears on the screen	Email field, password field, sign-in button, forgot-password link and register link.
User/Admin action	The user enters credentials and submits the form.
System behavior	Angular calls POST /api/auth/login. On success it stores the JWT token and role, then redirects according to the user role.

Table 22. Login page explanation

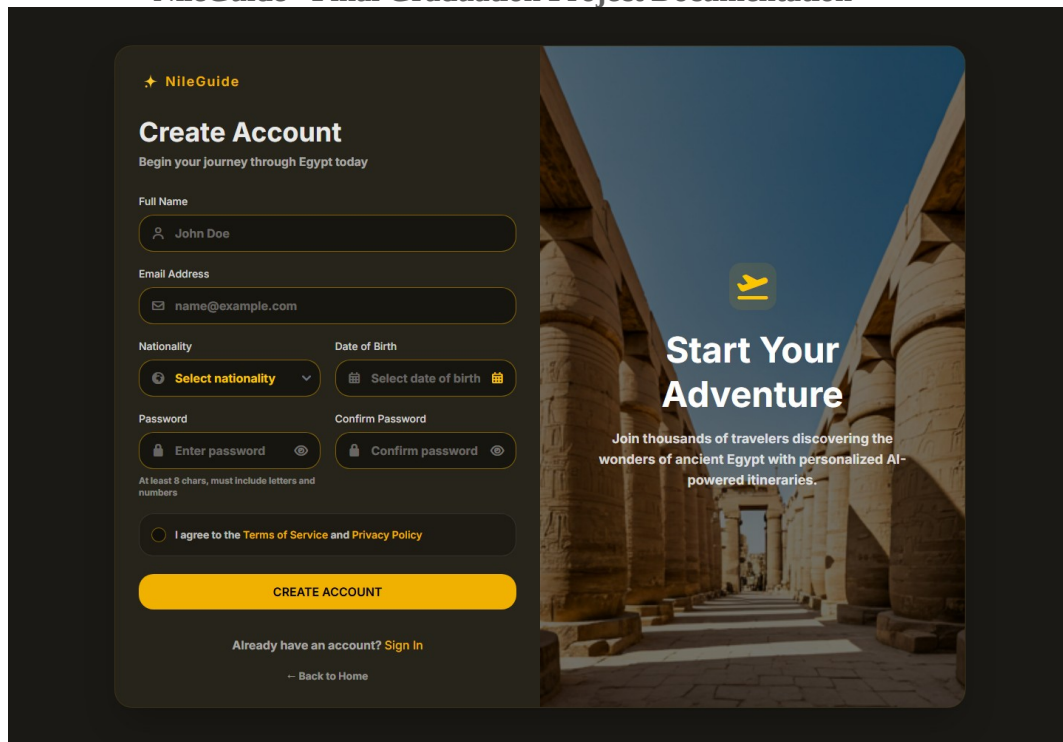


Figure 10. Register page

Purpose	Creates a new tourist account.
What appears on the screen	Full name, email, nationality, date of birth, password, confirmation password and agreement checkbox.
User/Admin action	A new tourist fills the form and creates an account.
System behavior	Angular validates the form and sends the registration payload to POST /api/auth/register.

Table 23. Register page explanation

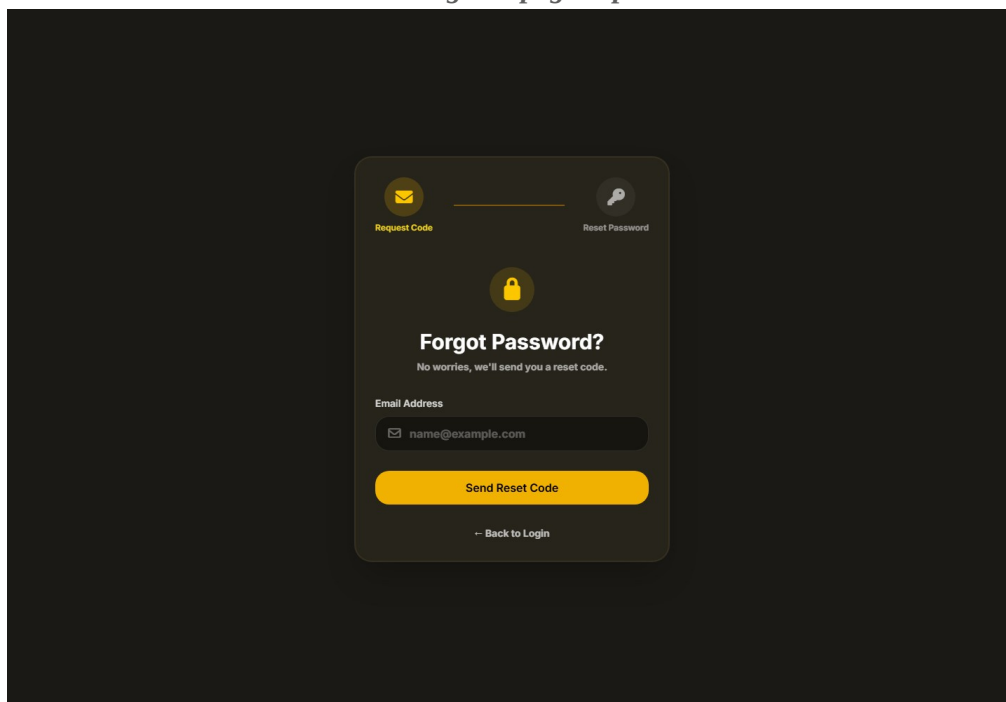


Figure 11. Forgot password page

Purpose	Starts the password recovery process.
What appears on the screen	Email input and reset request action.
User/Admin action	The user enters the email address used in the account.
System behavior	The backend handles password reset request and sends/validates reset information through the configured email flow.

Table 24. Forgot password page explanation

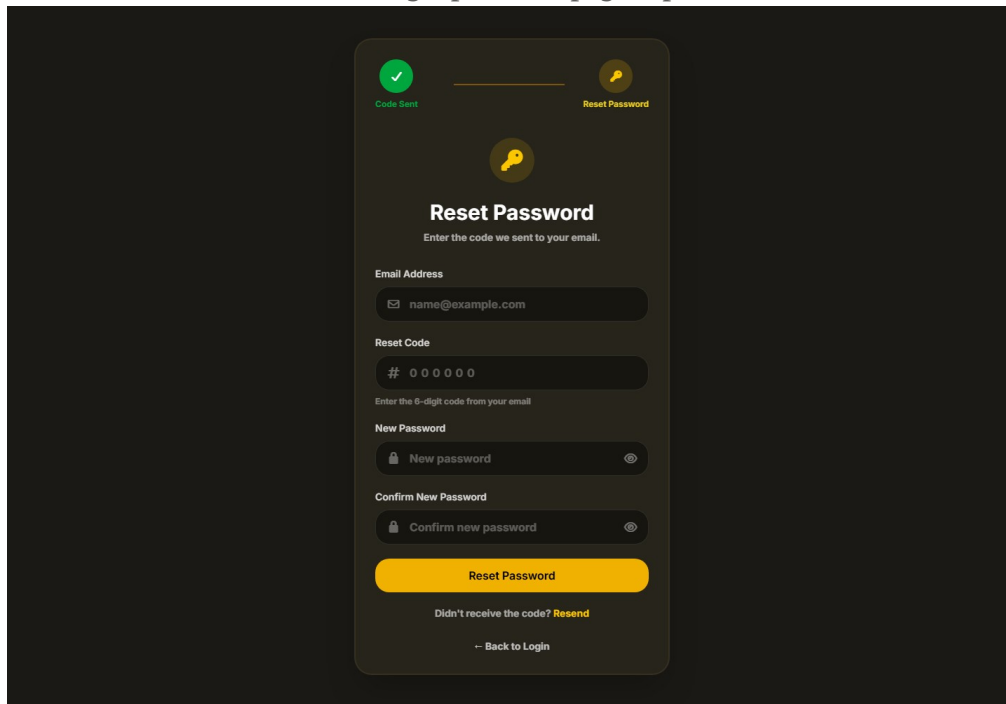


Figure 12. Reset password page

Purpose	Completes password recovery after receiving a reset code.
What appears on the screen	Email/code/password fields depending on reset flow and submit action.
User/Admin action	The user submits the verification/reset information.
System behavior	The frontend calls password-reset endpoints such as verify-reset-code and reset-password.

Table 25. Reset password page explanation

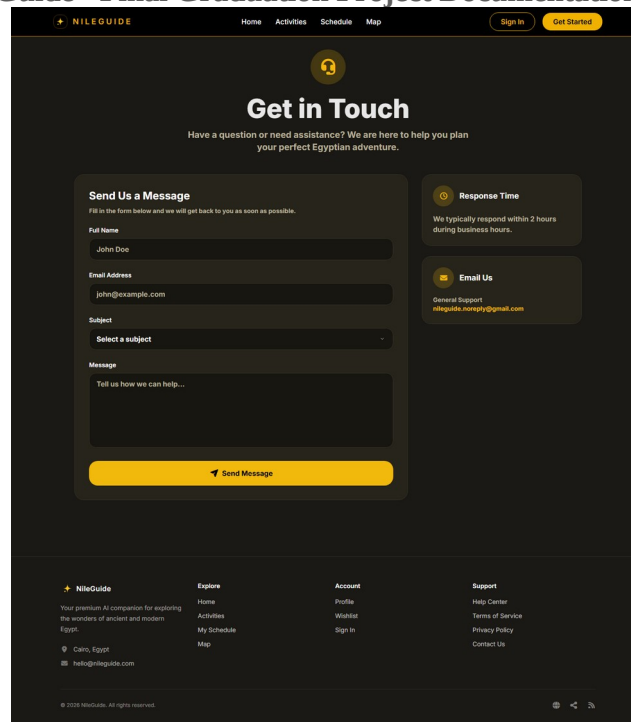


Figure 13. Contact page

Purpose	Allows visitors to contact the NileGuide team.
What appears on the screen	Contact form and support information.
User/Admin action	The visitor writes a message or uses the displayed contact information.
System behavior	The frontend handles message submission through configured contact/email integration where available.

Table 26. Contact page explanation

9.3 Tourist Feature Screens

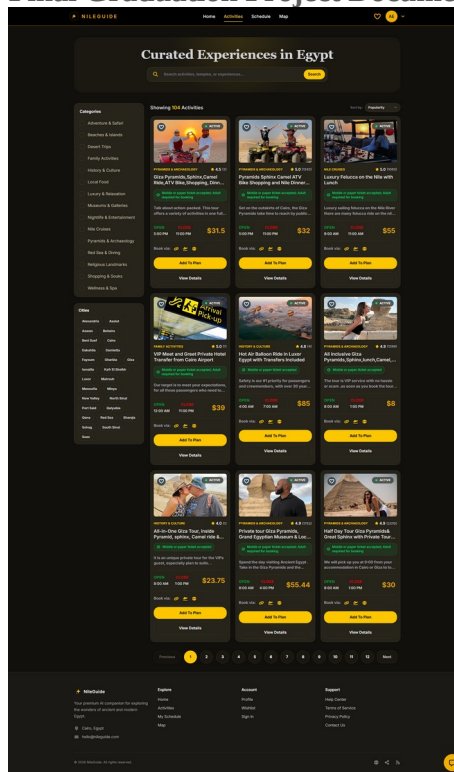


Figure 14. Activities page

Purpose	Main discovery page where tourists browse Egyptian activities.
What appears on the screen	Search bar, category/city/filter area, sorting controls and activity cards with images, price, location and action buttons.
User/Admin action	The tourist searches, filters, opens details, saves to wishlist or starts planning from activity cards.
System behavior	Angular loads activities from GET /api/Activities and uses city/category lookup APIs for filtering.

Table 27. Activities page explanation

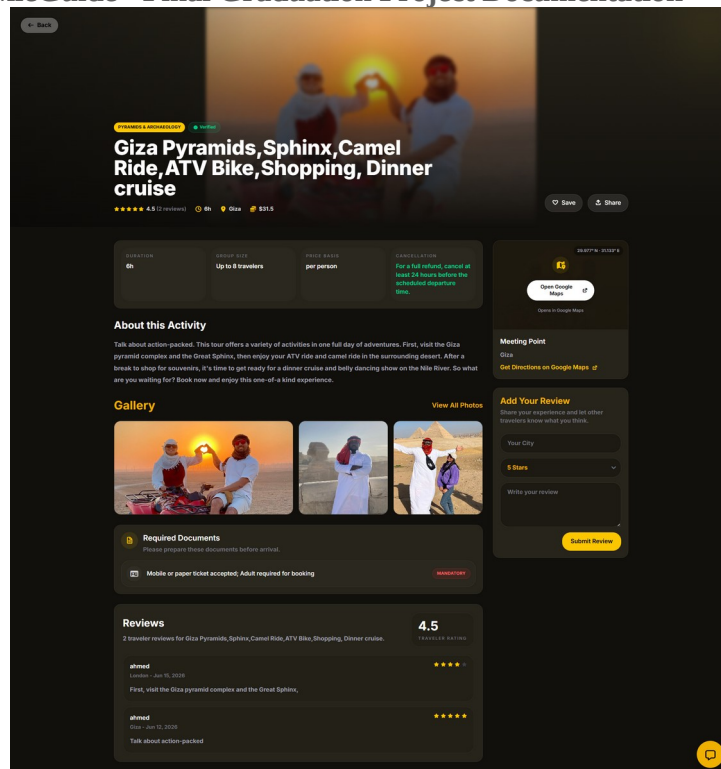


Figure 15. Activity details page

Purpose	Shows complete information about one selected activity.
What appears on the screen	Large hero image, save/share actions, summary cards, gallery, description, required documents, reviews, meeting point and add-review area.
User/Admin action	The tourist studies the activity, saves it, reviews it or decides whether to add it to the trip plan.
System behavior	The route /activities/:id loads details from GET /api/Activities/{id}; reviews use /api/activities/{activityId}/reviews.

Table 28. Activity details page explanation

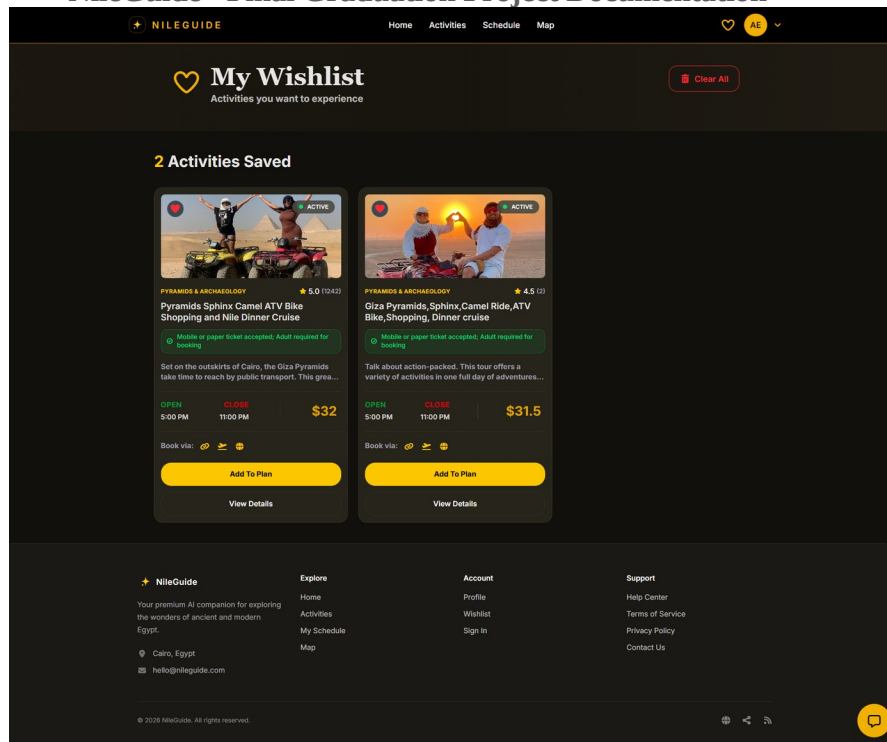


Figure 16. Wishlist page

Purpose	Collects activities that the tourist is interested in.
What appears on the screen	Saved activity cards, View Details, Add To Plan, remove actions and Clear All control.
User/Admin action	The tourist compares saved activities and decides which ones to plan.
System behavior	The page uses GET /api/wishlist, POST /api/wishlist/{activityId}, DELETE /api/wishlist/{activityId} and activity IDs for state.

Table 29. Wishlist page explanation

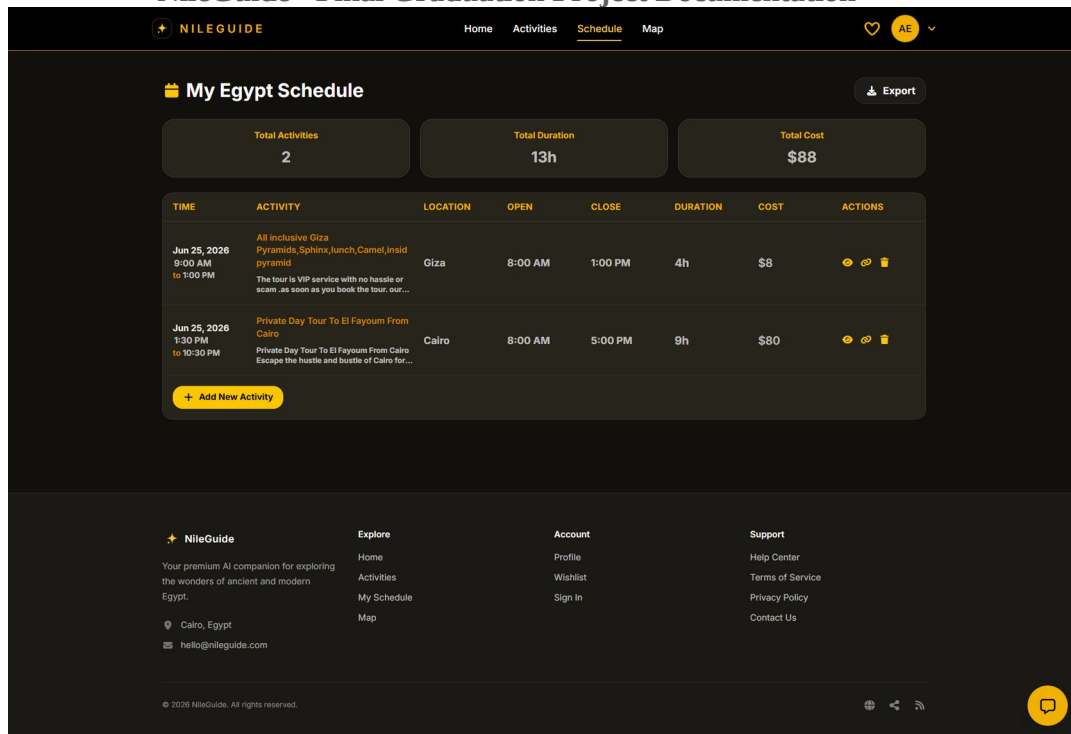


Figure 17. Schedule page

Purpose	Displays the tourist itinerary and trip summary.
What appears on the screen	Trip summary cards for total activities, duration and estimated cost, itinerary entries and export action.
User/Admin action	The tourist reviews the current plan and exports or manages itinerary items.
System behavior	The page loads plan items from GET /api/plan and removes items through DELETE /api/plan/items/{planItemId}.

Table 30. Schedule page explanation

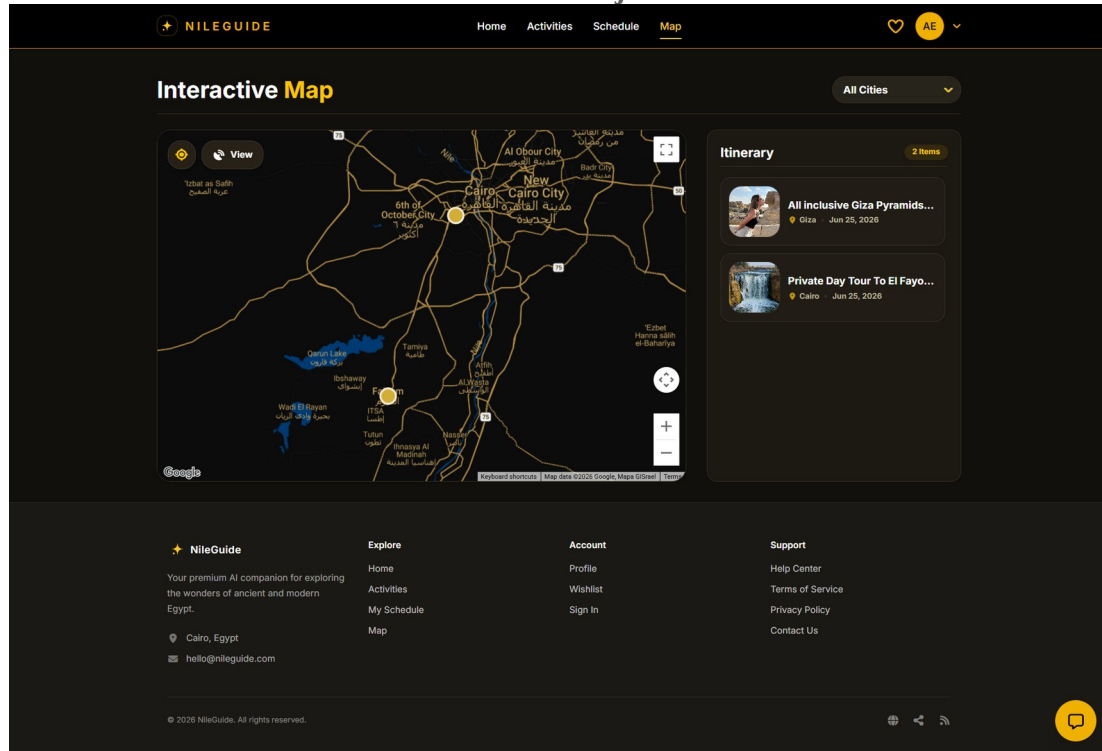


Figure 18. Map page

Purpose	Visualizes activities and itinerary locations on a map.
What appears on the screen	Google Map area, location markers, itinerary side panel, city filter and selected activity information.
User/Admin action	The tourist checks where the scheduled or available activities are located.
System behavior	The frontend uses activity latitude/longitude and map integration to render locations.

Table 31. Map page explanation

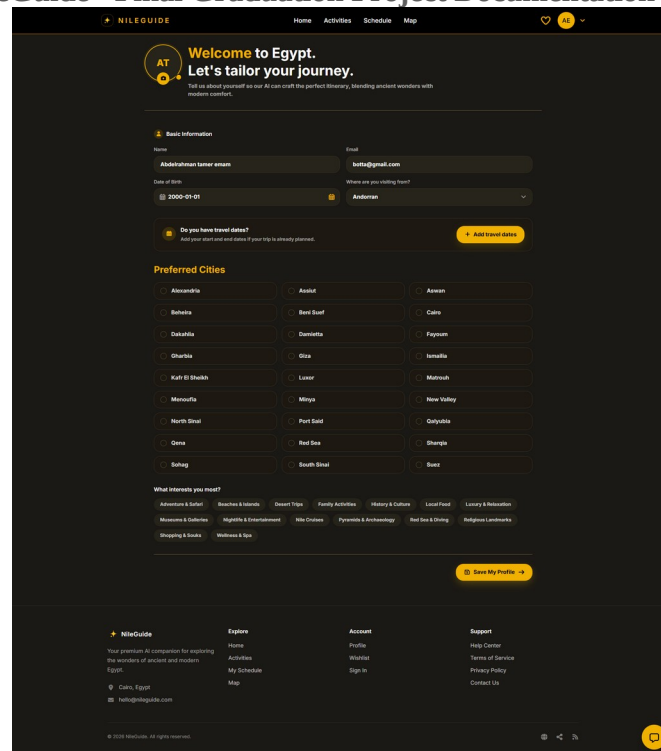


Figure 19. Profile page

Purpose	Allows the user to maintain personal data and travel preferences.
What appears on the screen	Avatar initials/profile picture, basic information, travel date section, preferred cities, interests and save action.
User/Admin action	The tourist updates profile information, chooses preferences and saves changes.
System behavior	The page uses GET/PUT /api/users/me/profile and profile-picture upload/delete endpoints.

Table 32. Profile page explanation

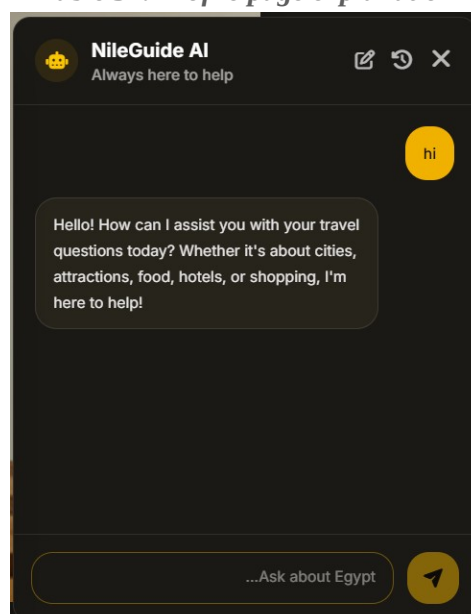


Figure 20. User chatbot panel

Purpose	Provides AI assistance from the user side.
What appears on the screen	Floating NileGuide AI panel with chat input and action controls.
User/Admin action	The user opens the assistant and asks tourism-related questions.
System behavior	The frontend sends the prompt to the FastAPI RAG chat endpoint and displays the returned answer.

Table 33. User chatbot panel explanation

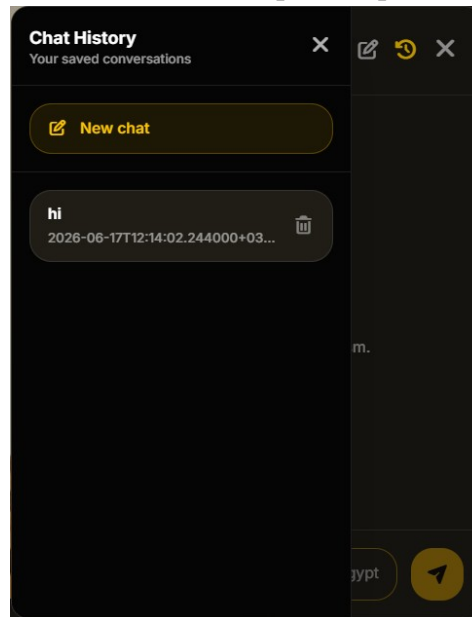


Figure 21. Chat history view

Purpose	Shows previous AI conversation entries where available.
What appears on the screen	Chat history panel listing saved or recent conversation records.
User/Admin action	The user can return to earlier conversations instead of starting from zero.
System behavior	Conversation metadata is handled through the .NET chat-conversations API while answers come from the RAG service.

Table 34. Chat history view explanation

9.4 Admin Feature Screens

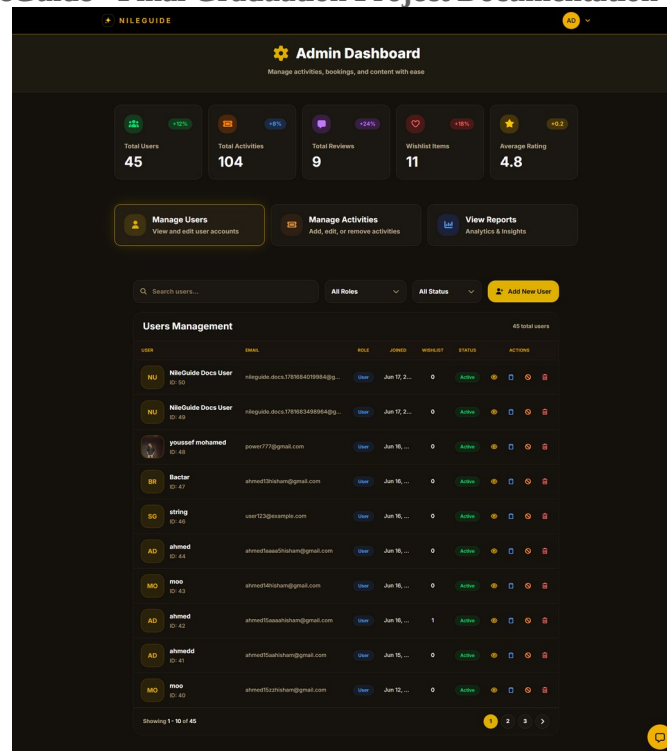


Figure 22. Admin users management

Purpose	Allows administrators to monitor and manage registered users.
What appears on the screen	Search, filters, user table/cards, role/status information and action controls.
User/Admin action	The admin views, filters and manages user accounts.
System behavior	The page uses /api/users endpoints such as list, get by id, update role, update status and delete.

Table 35. Admin users management explanation

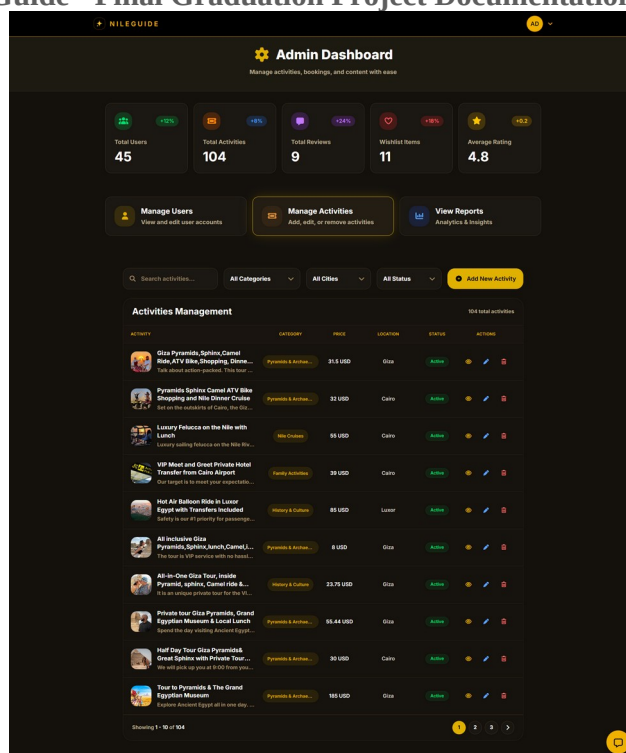


Figure 23. Admin activities management

Purpose	Allows administrators to maintain the activity catalogue.
What appears on the screen	Search, filters, activity table/list, active status, images and action buttons for view/update/delete.
User/Admin action	The admin adds new activities, edits existing activities or removes inactive/incorrect records.
System behavior	CRUD uses /api/admin/activities. Create and update use multipart/form-data to support uploaded images and related activity fields.

Table 36. Admin activities management explanation

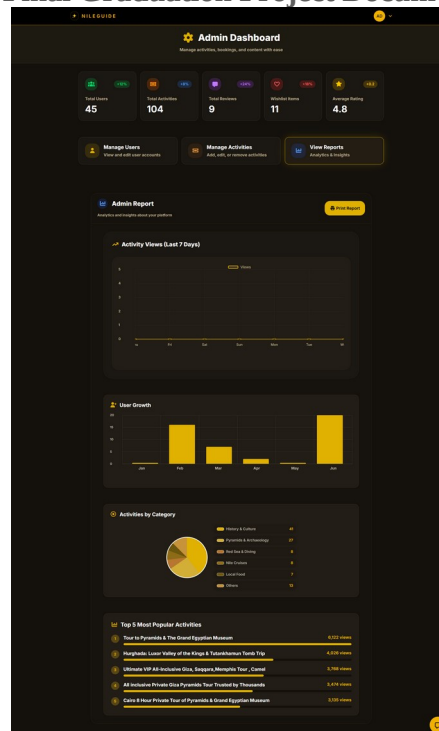


Figure 24. Admin reports page

Purpose	Presents operational analytics for the platform.
What appears on the screen	Charts and statistics related to activity views, user growth, categories and top activities.
User/Admin action	The admin reviews platform behavior and identifies popular content.
System behavior	The page uses /api/reports/activity-views, /user-growth, /activities-by-category and /top-activities.

Table 37. Admin reports page explanation

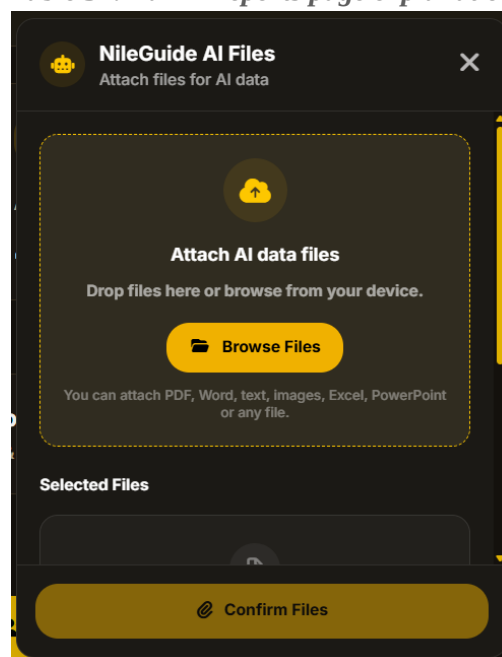


Figure 25. Admin AI files panel

Purpose	Allows administrators to update the AI assistant
----------------	--

	knowledge base.
What appears on the screen	Floating panel for attaching or uploading AI data files and managing the assistant content.
User/Admin action	The admin uploads files that should become part of the chatbot knowledge.
System behavior	The frontend calls the FastAPI data endpoints for upload, list, update and delete. The AI service indexes documents into MongoDB and Qdrant.

Table 38. Admin AI files panel explanation

Chapter 10 - API Documentation

10.1 API Design Overview

The main ASP.NET Core backend exposes REST endpoints under the /api prefix. Most tourist and admin features use JSON request/response bodies, while activity create/update and image operations use multipart/form-data. The AI service exposes a separate /api/v1 namespace for data upload and chat.

10.2 Main Backend Endpoints

Module	Endpoint	Purpose
Auth	POST /api/auth/register	Creates a new user account.
Auth	POST /api/auth/login	Authenticates and returns token/role data.
Auth	GET /api/auth/me	Returns current authenticated user information.
Auth	POST /api/auth/forgot-password	Starts password reset.
Auth	POST /api/auth/verify-reset-code	Verifies reset code.
Auth	POST /api/auth/reset-password	Sets a new password.
Activities	GET /api/Activities	Returns activity list with filters/pagination.
Activities	GET /api/Activities/{id}	Returns full dynamic activity details.
Categories	GET /api/Categories	Returns category lookup list.
Cities	GET /api/Cities	Returns city lookup list.
Wishlist	GET /api/wishlist	Returns authenticated user wishlist.
Wishlist	GET /api/wishlist/activity-ids	Returns activity IDs saved by the user.
Wishlist	POST /api/wishlist/{activityId}	Adds activity to wishlist.
Wishlist	DELETE /api/wishlist/{activityId}	Removes activity from wishlist.
Wishlist	GET /api/wishlist/status/{activityId}	Checks if an activity is wishlisted.
Plan	GET /api/plan	Returns user schedule/plan.
Plan	POST /api/plan/items	Adds an activity to schedule.
Plan	DELETE /api/plan/items/{planItemId}	Deletes a planned item.
Plan	GET /api/plan/activity-ids	Returns scheduled activity IDs.

Reviews	GET /api/activities/{activityId}/reviews	Returns reviews for one activity.
Reviews	POST /api/activities/{activityId}/reviews	Creates a review.
Profile	GET /api/users/me/profile	Returns current user profile.
Profile	PUT /api/users/me/profile	Updates current user profile.

Table 39. Main tourist/backend endpoints

10.3 Admin Backend Endpoints

Module	Endpoint	Purpose
Admin Activities	GET /api/admin/activities	Admin activity list.
Admin Activities	POST /api/admin/activities	Create activity with form-data and images.
Admin Activities	PUT /api/admin/activities/{id}	Update activity and optional images.
Admin Activities	DELETE /api/admin/activities/{id}	Delete activity.
Users	GET /api/users	List and filter users.
Users	GET /api/users/{id}	View user details.
Users	POST /api/users	Create user from admin side.
Users	PATCH /api/users/{id}/role	Change user role.
Users	PATCH /api/users/{id}/status	Change active/blocked status.
Users	DELETE /api/users/{id}	Delete user.
Dashboard	GET /api/dashboard	Dashboard summary statistics.
Reports	GET /api/reports/activity-views	Activity view analytics.
Reports	GET /api/reports/user-growth	User growth analytics.
Reports	GET /api/reports/activities-by-category	Category distribution.
Reports	GET /api/reports/top-activities	Top activity list.
Chat Conversations	GET /api/chat/conversations	Retrieve saved conversation metadata.
Chat Conversations	POST /api/chat/conversations	Save conversation metadata.
Chat Conversations	DELETE /api/chat/conversations/{conversationId}	Delete conversation metadata.

10.4 AI Service Endpoints

Endpoint	Method	Input	Output / purpose
/	GET	No body	Health/root response.
/api/v1/data/multi-upload	POST	One or more files as multipart/form-data.	Stores and indexes knowledge files.
/api/v1/data/files	GET	No body.	Lists uploaded AI knowledge files.
/api/v1/data/file/{file_id}	PUT	Replacement/updated file or metadata.	Updates a file and its indexed content.
/api/v1/data/file/{file_id}	DELETE	File id.	Removes file and related indexed chunks.
/api/v1/nlp/chat	POST	Question/prompt payload.	Returns RAG-based answer and supporting context where available.

Table 41. AI service endpoints

10.5 Request Handling Notes

- GET endpoints are used for lists, details, lookup data and reports.
- POST endpoints are used for creation operations such as register, login, wishlist add, plan add, reviews and AI upload/chat.
- PUT is used for full update operations such as profile and admin activity update.
- PATCH is used for targeted admin user changes such as role and status.
- DELETE is used for removing wishlist items, plan items, users, activities and AI files.
- Multipart/form-data is used when uploading images or AI files because binary data cannot be represented cleanly as plain JSON.

Chapter 11 - Testing and Validation

11.1 Testing Strategy

Testing was performed through a mix of manual UI testing, API testing, build checks and automated screenshot validation. Because NileGuide is a full-stack project, testing focused on the complete flow: the page should load, the service should call the correct API, the backend should return valid data, and the UI should update without breaking layout.

11.2 Manual Test Cases

ID	Scenario	Action	Expected result
TC-01	Register new tourist	Fill valid registration form.	Account is created or validation message appears for invalid data.
TC-02	Login tourist	Enter tourist credentials.	Token saved and tourist pages become accessible.
TC-03	Login admin	Enter admin credentials.	Admin dashboard and management modules become accessible.
TC-04	Browse activities	Open /activities and use filters.	Cards update according to search/filter values.
TC-05	Open activity details	Click an activity or navigate to /activities/:id.	Details, gallery and review area load dynamically.
TC-06	Wishlist add/remove	Save an activity then remove it.	Wishlist status changes and page updates.
TC-07	Schedule item	Add activity to plan and open schedule.	Activity appears in itinerary summary.
TC-08	Map visualization	Open map after schedule is available.	Markers/panel display planned activity locations.
TC-09	Profile update	Edit user fields and save.	Updated data persists when profile reloads.
TC-10	Admin activity management	Create/update/delete activity from admin page.	Activity catalogue reflects admin changes.
TC-11	Reports	Open reports screen.	Charts and report tables load without layout errors.
TC-12	AI upload and chat	Upload AI file and ask a question.	File is indexed and answer is generated from retrieved knowledge.

Table 42. Manual test cases

11.3 Screenshot Automation

A Playwright script was used to capture documentation screenshots for the important pages. The script logs in through the API using real tourist and admin accounts, captures public pages, tourist pages and admin pages, and writes a screenshot report. This helped validate that the routes can be opened and that the main screens are visually present.

11.4 Build and API Validation

- Frontend build validation uses npm run build or the Angular build pipeline to detect TypeScript and template errors.
- Backend validation uses Swagger/Postman-style API checks to confirm route behavior and payloads.
- Image upload flows were checked with multipart/form-data because activity and profile images are not simple JSON values.
- AI file upload was checked using supported file types such as PDF, DOCX, TXT and CSV.
- Protected-page testing included checking redirects or 401 responses when token data is missing or invalid.

11.5 Known Validation Notes

Area	Note	Impact
Refresh token / Remember Me	Not treated as final user-visible features.	Avoids presenting unconfirmed behavior during discussion.
Reports activity views	Report screens exist; activity-view logging should be verified with final backend data.	If views are empty, seed/logging data may be needed for demo.
AI conversation history	Chat answers are produced by FastAPI RAG; conversation metadata can be handled by .NET backend.	Demo should explain the split clearly.
Admin user creation payload	Needs consistency check if admin add-user sends multipart but API expects JSON.	Can be fixed or avoided in final demo if not needed.

Table 43. Validation notes and demo cautions

Chapter 12 - Deployment and Operations

12.1 Deployment Overview

NileGuide is designed so each part of the system can be deployed and maintained separately. The Angular frontend can be served as the public user interface. The ASP.NET Core backend can be hosted on Azure App Service and connected to Azure SQL. The AI/RAG service can run separately because it requires Python dependencies and vector-database connections.

Part	Deployment role	Operational notes
Frontend	Public web application.	Needs correct API base URLs and production build.
ASP.NET Core API	Main business backend.	Needs JWT settings, database connection, CORS and Cloudinary/email settings.
SQL Server / Azure SQL	Relational data storage.	Needs migrations/seed data and secure connection strings.
Cloudinary	Image storage.	Needs cloud name, API key/secret and upload settings.
FastAPI RAG service	AI data and chat service.	Needs MongoDB, Qdrant and model-provider environment variables.
MongoDB	AI metadata/chunks.	Stores uploaded file metadata and extracted chunks.
Qdrant	Vector storage.	Stores embeddings used during retrieval.

Table 44. Deployment components

12.2 Environment Configuration

- Frontend environment contains the public backend API base URL and AI service base URL where needed.
- Backend environment contains database connection string, JWT settings, Cloudinary settings and email settings.
- AI environment contains MongoDB connection, Qdrant settings and LLM/embedding model provider keys.
- Secrets such as database passwords and API keys should never be stored directly in public repositories.
- CORS must allow the deployed frontend domain to call the backend and AI services.

12.3 Operation and Maintenance

After deployment, the administrator can maintain most business content without redeploying the frontend. Activities can be managed through the admin dashboard, images can be uploaded through the browser, and AI knowledge can be updated by uploading new files. This operational model is one of the important differences between a static graduation website and a maintainable platform.

Chapter 13 - Challenges and Solutions

13.1 Main Implementation Challenges

Challenge	Cause	Solution / handling
JWT protected routes after refresh	Protected pages need token data before API calls.	Use stored access token and guards/interceptors; do not document Refresh/Remember Me as delivered scope.
SSR localStorage errors	Angular SSR can run code where browser-only APIs do not exist.	Use platform/browser checks before localStorage access.
Dynamic activity details	The details page must load data by ID instead of static content.	Use route /activities/:id and GET /api/Activities/{id}.
Image upload	Images cannot be sent as normal JSON strings.	Use multipart/form-data and Cloudinary storage.
Admin activity update response	Update endpoint may not return the full updated object.	Refresh the list or update UI state after successful response.
AI grounding	Generic answers can hallucinate.	Use RAG retrieval from uploaded knowledge files.
Screenshot capture/login	UI login automation may fail with protected pages.	Use API login in the Playwright screenshot script and real credentials.

Table 45. Challenges and solutions

13.2 Lessons Learned

- A full-stack project needs agreement between frontend payloads and backend DTOs before UI work is finished.
- Documentation should follow the real delivered scope, not only the first-term plan.
- Admin tools are important because tourism content changes over time.
- RAG is easier to explain and maintain when file upload, chunking, vector storage and chat are documented separately.
- Screenshots are more useful when each image is followed by a direct explanation of user action and system behavior.

Chapter 14 - Conclusion and Future Work

14.1 Conclusion

NileGuide delivers a complete tourism planning platform that connects activity discovery with practical planning tools. The tourist can browse activities, view details, save a wishlist, build a schedule, view the map, update profile preferences and ask the AI assistant. The administrator can manage users, activities, reports and AI knowledge files. The architecture separates the Angular frontend, ASP.NET Core backend and FastAPI RAG service, making the system understandable and maintainable.

The final project evolved from the midyear concept into a stronger implementation. Instead of documenting planned features as if they were finished, this report focuses on the actual delivered modules and clearly identifies future work. This makes the project safer to discuss in a graduation defense because every described module is tied to a screen, API, database entity or service flow.

14.2 Future Work

- Add real booking confirmation and payment integration with external providers.
- Add a user-visible Remember Me and refresh-token session flow after completing and testing the full auth lifecycle.
- Improve AI trip-planning so the assistant can generate structured multi-day itineraries using real activity availability and user preferences.
- Add stronger recommendation algorithms using user behavior, ratings, preferred cities and categories.
- Add multi-language support for Arabic/English tourism content.
- Add notifications for schedule reminders, changed activity status or admin announcements.
- Improve reports with more reliable activity-view logging, export options and date range filters.
- Add mobile application support or PWA enhancements for tourists traveling on the go.

Chapter A - Appendices

Appendix A - Feature-to-API Traceability

Feature	UI location	API	Main data
Login/Register	Auth pages	POST /api/auth/login, POST /api/auth/register	Users
Activities list	Activities page	GET /api/Activities	Activities, Cities, Categories
Activity details	Activity details page	GET /api/Activities/{id}	Activities, Images, Hours, BookingLinks, Reviews
Wishlist	Wishlist page/cards	GET/POST/DELETE /api/wishlist	WishlistItems
Schedule	Schedule page	GET/POST/DELETE /api/plan	PlanItems
Map	Map page	GET /api/plan + activity coordinates	Activities, PlanItems
Profile	Profile page	GET/PUT /api/users/me/profile	UserProfiles
Admin users	Users management	GET/PATCH/ DELETE /api/users	Users
Admin activities	Activities management	GET/POST/PUT/ DELETE /api/admin/activities	Activities, Images, Hours, BookingLinks
Reports	Reports page	GET /api/reports/*	Users, Activities, ActivityViews, Reviews
AI files	Admin AI panel	POST/GET/PUT/ DELETE /api/v1/data/*	MongoDB/Qdrant AI storage
AI chat	Chatbot panel	POST /api/v1/nlp/chat	MongoDB/Qdrant retrieval + LLM

Table 46. Feature-to-API traceability

Appendix B - Demo Checklist

- Open Home and explain the purpose of NileGuide.
- Login as tourist and browse Activities.
- Open one Activity Details page and explain dynamic API data.
- Add an activity to Wishlist and open Wishlist.
- Add an activity to Schedule and open Schedule/Map.
- Open Profile and explain preferences/travel dates/profile picture.
- Open User Chatbot and ask a simple question.
- Logout or clear state, then login as Admin.

- Open Users Management and explain role/status operations.
- Open Activities Management and explain create/update/delete with images.
- Open Reports and explain what each chart measures.
- Open AI Files panel and explain upload/index/chat RAG flow.

Appendix C - Backend DTO and Payload Examples

The following examples describe the main data contracts at a documentation level. They are not intended to replace Swagger, but they help the discussion panel understand what each important request contains.

Operation	Typical request data	Important response / behavior
Login	email, password	Returns JWT access token, user id and role. The frontend stores the token and redirects according to role.
Register	fullName, email, password, nationality, dateOfBirth	Creates a tourist account and returns authentication information when registration succeeds.
Update profile	fullName, email, nationality, dateOfBirth, hasTravelDates, travelStartDate, travelEndDate, preferredCityIds, interestCategoryIds	Updates personal preferences used by the user profile and activity filtering experience.
Create activity	multipart/form-data including name, description, city, category, price, duration, location, images, booking links and hours	Admin creates a new activity and uploaded images are stored externally.
Add to wishlist	activityId in route	Creates a relationship between the authenticated user and the selected activity.
Add plan item	activityId, selected date/time and related scheduling values	Adds the activity to the authenticated user itinerary.
Create review	rating and comment for a selected activity	Stores user feedback and contributes to activity review display.
AI chat	question/prompt and optional conversation context	RAG service retrieves relevant chunks and returns an answer.

Table 47. Important request/response contracts

Appendix D - Frontend Component Map

Screen / component	Main UI responsibility	Connected services
Navbar	Navigation, login state, avatar/profile access and links.	Auth/Profile state where available.
Home page	Landing sections, city cards, how-	Activities lookup for featured

	it-works and popular activities preview.	content where connected.
Activities component	Cards, filters, sorting, pagination/search and activity actions.	ActivitiesService, WishlistService, Plan/Schedule service.
Activity details component	Hero, gallery, description, documents, reviews and location information.	ActivitiesService, WishlistService, ReviewsService.
Wishlist component	Saved activities list and actions to remove, clear or add to plan.	WishlistService, Plan/Schedule service.
Schedule component	Itinerary summary, planned items and export controls.	Plan/Schedule service, jsPDF export utilities.
Map component	Google Map view with itinerary panel and markers.	Plan/Schedule service and activity coordinates.
Profile component	User information, travel dates, preferences and profile picture.	ProfileService, lookup services.
Admin users component	User table/list, search, filters and admin actions.	UserService.
Admin activities component	Activity table/list, modals, image upload and CRUD actions.	Admin ActivityService, lookup services.
Reports component	Charts and report cards for dashboard analytics.	ReportsService, ApexCharts.
Admin chatbot/files component	AI file upload and file-management panel.	AI/RAG service endpoints.

Table 48. Frontend component responsibilities

Appendix E - UI/UX Design System Notes

The visual design uses a dark background with yellow/gold accents. This matches the tourism/adventure identity while keeping cards and buttons visible. The design is intentionally card-based because tourism activities depend heavily on images, short highlights and quick actions.

Design element	Usage	Reason
Dark background	Main page backgrounds and panels.	Improves contrast for gold accents and activity images.
Gold/yellow accents	Primary buttons, active states and important highlights.	Creates a recognizable NileGuide visual identity.
Cards	Activities, wishlist items, statistics and summaries.	Makes content easy to scan and compare.
Modals	Admin add/update/delete flows.	Keeps the admin inside the same management page while performing actions.

Tables	Admin users, activities, endpoints and documentation data.	Suitable for structured operational information.
Floating chatbot	User and admin AI access.	Keeps AI available without forcing navigation away from the current page.

Table 49. UI design system decisions

Appendix F - Error Handling and Loading Behavior

- Public pages should load even when the user is not authenticated.
- Protected pages should redirect or block access when the token is missing or invalid.
- API errors should not leave the user on a blank screen; the UI should display an error or empty state.
- Admin modals should close or refresh the list only after a successful API response.
- File upload errors should be shown clearly because upload failures may be caused by file size, unsupported type or server validation.
- AI chat should handle empty answers, failed retrieval or unavailable AI service without breaking the page.
- SSR-related code must avoid direct browser API access unless the platform is confirmed to be browser.

Appendix G - Final Demo Flow

Step	Screen	What to show	Key explanation
1	Home	Landing content and project purpose.	NileGuide is not a static brochure; it connects discovery and planning.
2	Login	Tourist login.	JWT token and role control protected access.
3	Activities	Filters and activity cards.	Data is loaded from backend APIs, not hard-coded.
4	Activity details	Gallery, documents, reviews and location.	Route /activities/:id is fully dynamic.
5	Wishlist	Saved activities and actions.	Wishlist is user-specific and stored through the backend.
6	Schedule	Trip summary and itinerary.	Plan items connect activities with user planning.
7	Map	Markers and itinerary panel.	Location data is used visually to support planning.
8	Profile	Preferences and picture.	Profile preferences can support better filtering and personalization.
9	Admin users	User management.	Admin role controls

			operational access.
10	Admin activities	CRUD and image upload.	Activities can be maintained without changing frontend code.
11	Reports	Charts and statistics.	Reports convert database data into admin insights.
12	AI files/chatbot	Upload knowledge file and ask question.	RAG grounds answers in uploaded documents.

Table 50. Recommended final demo flow

Appendix H - Expanded Data Dictionary

Data object	Used by	Notes
ActivityListItem	Activities, wishlist and admin listing screens.	Contains concise card data such as name, image, price, city/category and status.
ActivityDetails	Activity details page.	Contains long description, image gallery, booking links, opening hours, documents and reviews.
UserProfileResponse	Profile page and navbar avatar state.	Contains full name, email, nationality, date of birth, travel dates, preferences and picture URL.
WishlistPagedResponse	Wishlist page.	Returns saved activities and paging/list metadata.
PlanItem	Schedule and map pages.	Represents one activity selected by a user for a date/time.
DashboardStatsResponse	Admin dashboard.	Aggregates counts and summary values.
ReportItem DTOs	Reports page.	Shape chart data for activity views, user growth, category distribution and top activities.
AI File Metadata	Admin AI files panel.	Stores uploaded file id, name, size/status and indexing metadata.
Chat Request/Response	User/admin chatbot.	Sends user prompt and returns answer generated from retrieved context.

Table 51. Expanded data dictionary

Appendix C - References

- [1] Microsoft ASP.NET Core Documentation.
- [2] Angular Documentation.
- [3] Entity Framework Core Documentation.

- [4] SQL Server Documentation.
- [5] FastAPI Documentation.
- [6] MongoDB Documentation.
- [7] Qdrant Documentation.
- [8] OpenAI API / Embeddings documentation.
- [9] Cloudinary Documentation.
- [10] Tailwind CSS Documentation.
- [11] Flowbite Documentation.
- [12] Google Maps Platform Documentation.

Appendix D - Figures and Tables Index

- Figure 1. Final NileGuide architecture
- Figure 2. Tourist workflow
- Figure 3. Admin workflow
- Figure 4. JWT authentication sequence
- Figure 5. Deployment architecture
- Figure 6. Simplified ERD for NileGuide
- Figure 7. RAG workflow for NileGuide AI assistant
- Figure 8. Home page
- Figure 9. Login page
- Figure 10. Register page
- Figure 11. Forgot password page
- Figure 12. Reset password page
- Figure 13. Contact page
- Figure 14. Activities page
- Figure 15. Activity details page
- Figure 16. Wishlist page
- Figure 17. Schedule page
- Figure 18. Map page
- Figure 19. Profile page
- Figure 20. User chatbot panel
- Figure 21. Chat history view
- Figure 22. Admin users management
- Figure 23. Admin activities management
- Figure 24. Admin reports page
- Figure 25. Admin AI files panel
- Table 1. NileGuide team members
- Table 2. List of abbreviations
- Table 3. Problem definition and project response
- Table 4. Final project scope
- Table 5. Final project contributions
- Table 6. Related work comparison
- Table 7. System stakeholders
- Table 8. Functional requirements
- Table 9. Non-functional requirements
- Table 10. System layers and technologies
- Table 11. Database entity dictionary
- Table 12. Backend technology stack
- Table 13. Backend organization
- Table 14. Backend controllers

Table 15. Backend feature processing summary

Table 16. Frontend technology stack

Table 17. Frontend route map

Table 18. Frontend services and responsibilities

Table 19. AI/RAG technology stack

Table 20. AI/RAG endpoints

Table 21. Home page explanation

Table 22. Login page explanation

Table 23. Register page explanation

Table 24. Forgot password page explanation

Table 25. Reset password page explanation

Table 26. Contact page explanation

Table 27. Activities page explanation

Table 28. Activity details page explanation

Table 29. Wishlist page explanation

Table 30. Schedule page explanation

Table 31. Map page explanation

Table 32. Profile page explanation

Table 33. User chatbot panel explanation

Table 34. Chat history view explanation

Table 35. Admin users management explanation

Table 36. Admin activities management explanation

Table 37. Admin reports page explanation

Table 38. Admin AI files panel explanation

Table 39. Main tourist/backend endpoints

Table 40. Admin and reporting endpoints

Table 41. AI service endpoints

Table 42. Manual test cases

Table 43. Validation notes and demo cautions

Table 44. Deployment components

Table 45. Challenges and solutions

Table 46. Feature-to-API traceability

Table 47. Important request/response contracts

Table 48. Frontend component responsibilities

Table 49. UI design system decisions

Table 50. Recommended final demo flow

Table 51. Expanded data dictionary